

Os Principais Serviços de Arquivo (Syscalls)

Aqui estão os serviços essenciais que você precisa para criar, abrir, ler, escrever e fechar arquivos:

1. `sys_open` (Abrir ou Criar Arquivo)

Serve para abrir um arquivo existente ou criar um novo.

- **Número (RAX):** 2
- **Argumentos:**
 - `RDI`: Endereço da string com o caminho do arquivo (terminada em `0` / byte nulo).
 - `RSI`: Flags de acesso (Modo de abertura).
 - `0` = `O_RDONLY` (Apenas leitura)
 - `1` = `O_WRONLY` (Apenas escrita)
 - `2` = `O_RDWR` (Leitura e escrita)
 - `64` (ou `0o100` em octal) = `O_CREAT` (Cria o arquivo se ele não existir)
 - `RDX`: Permissões do arquivo (apenas se for criar um novo, ex: `0o644` para leitura/escrita do dono).
- **Retorno (RAX):** Retorna o **File Descriptor (FD)**, que é um número ID único para identificar aquele arquivo nas próximas funções. Se for um número negativo, indica um erro.

2. `sys_read` (Ler do Arquivo)

Lê dados a partir do arquivo aberto para um buffer de memória RAM.

- **Número (RAX):** 0
- **Argumentos:**
 - `RDI`: O File Descriptor (FD) obtido na abertura.
 - `RSI`: Endereço do buffer na memória onde os dados lidos serão guardados.
 - `RDX`: Número máximo de bytes que deseja ler.
- **Retorno (RAX):** O número real de bytes lidos com sucesso. Se retornar 0, significa que chegou ao fim do arquivo (EOF).

3. `sys_write` (Escrever no Arquivo)

Escreve dados de um buffer de memória diretamente para o arquivo.

- **Número (RAX):** 1
- **Argumentos:**
 - `RDI`: O File Descriptor (FD) do arquivo (ou 1 para escrever direto no terminal - stdout).
 - `RSI`: Endereço do buffer contendo o texto ou dados a serem gravados.
 - `RDX`: Quantidade de bytes que serão escritos.
- **Retorno (RAX):** O número de bytes que foram gravados com sucesso.

4. `sys_close` (Fechar o Arquivo)

Fecha o arquivo e libera o File Descriptor associado a ele. É uma boa prática obrigatória.

- **Número (RAX):** 3
 - **Argumentos:**
 - **RDI:** O File Descriptor (FD) que será fechado.
-

Exemplo Prático: Criando e Escrevendo em um Arquivo

Abaixo está um exemplo completo em NASM pronto para ser compilado no Linux de 64 bits. Ele cria um arquivo chamado `exemplo.txt` e escreve uma mensagem dentro dele.

```
section .data
    filename db "exemplo.txt", 0
    conteudo db "Ola! Escrevendo em arquivo via Assembly no Linux x64.", 10
    conteudo_len equ $ - conteudo

section .bss
    fd_arquivo resq 1          ; Reserva 8 bytes (quadword) para guardar o File
                                Descriptor

section .text
    global _start

_start:
    ; 1. ABRIR / CRIAR O ARQUIVO (sys_open)
    mov rax, 2                ; código da syscall sys_open
    mov rdi, filename          ; nome do arquivo
    mov rsi, 65                ; Flags: O_WRONLY (1) + O_CREAT (64) = 65 (Escrita
+ Cria se não existir)
    mov rdx, 0o644             ; Permissões em octal (rw-r--r--)
    syscall

    ; Salva o File Descriptor retornado em RAX na nossa variável
    mov [fd_arquivo], rax

    ; 2. ESCREVER NO ARQUIVO (sys_write)
    mov rax, 1                ; código da syscall sys_write
    mov rdi, [fd_arquivo]     ; passa o File Descriptor salvo
    mov rsi, conteudo          ; endereço da mensagem
    mov rdx, conteudo_len      ; tamanho da mensagem
    syscall

    ; 3. FECHAR O ARQUIVO (sys_close)
    mov rax, 3                ; código da syscall sys_close
    mov rdi, [fd_arquivo]     ; passa o File Descriptor
    syscall

    ; 4. ENCERRAR O PROGRAMA (sys_exit)
    mov rax, 60               ; código da syscall sys_exit
    mov rdi, 0                ; status de saída 0 (sem erros)
    syscall
```

Como compilar e rodar no Linux:

No seu terminal, execute os três comandos abaixo:

```
nasm -f elf64 código.asm -o código.o
ld código.o -o programa
./programa
```

Para ler os dados de um arquivo existente e exibi-los no terminal usando o **NASM no Linux de 64 bits**, nós combinamos as syscalls `sys_open` (para abrir), `sys_read` (para ler o conteúdo para a memória), `sys_write` (para jogar esse conteúdo no terminal através do `stdout`) e `sys_close` (para fechar o arquivo).

Exemplo Prático: Lendo e Imprimindo um Arquivo

O exemplo abaixo abre um arquivo chamado `texto.txt`, lê o seu conteúdo em blocos usando um buffer temporário e imprime tudo na tela.

```
section .data
    filename db "texto.txt", 0

section .bss
    fd_arquivo resq 1      ; Reserva 8 bytes para salvar o File Descriptor
    buffer resb 512        ; Cria um buffer temporário de 512 bytes para
leitura

section .text
    global _start

_start:
    ; 1. ABRIR O ARQUIVO EXISTENTE (sys_open)
    mov rax, 2             ; syscall número 2 (sys_open)
    mov rdi, filename       ; nome do arquivo
    mov rsi, 0              ; Flag: O_RDONLY (0 = Apenas Leitura)
    syscall

    ; Verifica se houve erro na abertura (se RAX for negativo)
    cmp rax, 0
    jl erro_sair            ; se menor que 0, desvia para o fim do programa

    mov [fd_arquivo], rax   ; Salva o File Descriptor retornado

ler_bloco:
    ; 2. LER O CONTEÚDO DO ARQUIVO (sys_read)
    mov rax, 0             ; syscall número 0 (sys_read)
    mov rdi, [fd_arquivo]   ; File Descriptor do arquivo aberto
    mov rsi, buffer         ; endereço do nosso buffer de memória
    mov rdx, 512            ; ler no máximo 512 bytes por vez
    syscall

    ; RAX agora contém o número real de bytes lidos com sucesso
    cmp rax, 0
    jle fechar_arquivo     ; Se for 0 (Fim do Arquivo / EOF) ou menor (Erro),
para de ler

    ; Guarda temporariamente a quantidade de bytes lidos em R12 para o
sys_write
    mov r12, rax

    ; 3. IMPRIMIR O BUFFER NO TERMINAL (sys_write no stdout)
    mov rax, 1             ; syscall número 1 (sys_write)
    mov rdi, 1              ; File Descriptor 1 = stdout (Terminal)
    mov rsi, buffer         ; endereço do conteúdo que acabamos de ler
    mov rdx, r12            ; quantidade exata de bytes que foram lidos
    syscall
```

```
    jmp ler_bloco          ; volta para ler o próximo bloco do arquivo (loop)

fechar_arquivo:
    ; 4. FECHAR O ARQUIVO (sys_close)
    mov rax, 3             ; syscall número 3 (sys_close)
    mov rdi, [fd_arquivo]
    syscall

erro_sair:
    ; 5. ENCERRAR O PROGRAMA (sys_exit)
    mov rax, 60            ; syscall número 60 (sys_exit)
    mov rdi, 0             ; status de retorno 0
    syscall
```

Como testar no seu Linux

Crie um arquivo de texto qualquer no mesmo diretório para servir de teste:

```
echo "Olá do arquivo texto.txt! O código em Assembly funcionou." > texto.txt
```

Compile e linke o código NASM:

```
nasm -f elf64 leitor.asm -o leitor.o
ld leitor.o -o leitor
```

Execute o programa:

```
./leitor
```

Detalhe importante sobre o Loop

Repare que o tamanho do buffer foi definido como **512 bytes**. Se o seu arquivo tiver mais do que 512 bytes, o programa não vai quebrar: ele simplesmente lê os primeiros 512 bytes, imprime, e o comando `jmp ler_bloco` faz ele ler os próximos 512 bytes automaticamente até que o arquivo termine (`RAX = 0`).